Programming Assignment 1

Distributed Graph Representation and Detecting Network Communities

Due: September 24, 2026 by 11:59PM
Submission: via Canvas, individual submission

1. Learning Objectives

By completing this assignment, you should be able to:

- Represent and analyze a large, sparse graph with Spark RDDs.
- Translate graph formulas into distributed joins and aggregations.
- Implement a local community detection in Spark.

The concepts and datasets are from Yang and Leskovec's SIGKDD paper [1].

2. Background

Graph data structure is suitable to represent various natural phenomena in technological, scientific, social, and information systems. Often, vertices in such graph organize into densely linked groups that are commonly referred to as *network communities* because of the distinctive interactions and associations among the vertices.

Key concept #1. Edges and Vertices in the DBLP network

In mathematical notation, a graph is written as:

$$G = (V, E)$$

where V is the set of vertices and E is the set of edges.

Vertices (nodes) refers to the objects or entities in the graph and *edges* (links) are the connections or relationships between pairs of vertices.

For the DBLP collaboration network, each vertex represents an author, and an edge connects two authors if they have coauthored at least one paper.

The included DBLP network dataset is an undirected graph. In other words, regardless of each author's contribution, all coauthors are represented based on their co-authorship, and each collaboration on a paper is recorded as an edge without direction.

Key concept #2. Weakly Connected Components

A *weakly connected component* (WCC) is a maximal group of vertices in a directed graph that are connected when the directions of all edges are ignored. In the DBLP network dataset, you should simply ignore the "*from*" and "*to*" and consider them as undirected edges. In an undirected graph, such as the DBLP co-authorship network, a weakly connected component is simply called a connected component. Every pair of authors in that component is connected through some sequence of co-authorship relationships.

A component is maximal, meaning that no additional vertex can be added while preserving connectivity.

Key concept #3. Average clustering coefficient

The *average clustering coefficient* measures how strongly the neighbors of vertices in a graph tend to be connected to one another.

For a vertex v with degree v_k, its local clustering coefficient is,

$$C_v = \frac{2e_v}{v_k(v_k - 1)}$$

Where e_v is the number of edges between the neighbors of v.

Equivalently,

$$C_v = \frac{number\ of\ connections\ among\ v's\ neighbors}{maximum\ possible\ number\ of\ those\ connections}$$

The average clustering coefficient of graph $G = (V, E)$ is

$$C_{avg} = \frac{1}{|V|}\sum_{v \in V} C_v$$

Vertices with fewer than two neighbors are typically assigned a clustering coefficient of zero.

For example, suppose author A has three coauthors. Those three coauthors could have at most three connections among themselves:

- If all three have collaborated with one another, $C_A = 1$.
- If only one pair has collaborated, $C_A = 1/3$.
- If none has collaborated, $C_A = 0$.

Therefore, the DBLP network's reported average clustering coefficient of 0.6324 indicates substantial local clustering: an author's coauthors are frequently also coauthors of one another.

Key Concept #4. Triangles in a Network

The DBLP network contains 2,224,385 triangles. A triangle consists of three authors who are pairwise connected. In the DBLP co-authorship network, authors *A*, *B*, and *C* form a triangle when all three co-authorship edges exist:

$$(A, B), (B, C), (A, C)$$

The number of triangles can be calculated using:

$$T = \frac{1}{3}\sum_{v \in V} t_v$$

Where t_v is the number of connected pairs among the neighbors of vertex *v*. Division by three is necessary because each triangle is counted once at each of its three vertices.

Counting triangles helps reveal a network's local cohesion and community structure. A triangle occurs when three vertices are pairwise connected. In the DBLP coauthorship network, a triangle indicates three authors for whom every pair has collaborated. This may represent a closely connected research group.

Triangle counts are useful for:

- Measuring clustering: More triangles generally indicate that neighbors frequently connect with one another.
- Identifying communities: Dense communities often contain many triangles.
- Evaluating collaboration patterns: In DBLP, triangles may reveal cohesive groups of researchers.
- Detecting important vertices: Authors participating in many triangles may be central members of research groups.
- Comparing networks: Triangle frequency can distinguish tightly clustered networks from sparse or randomly structured networks.
- Detecting anomalies: An unexpectedly high or low number of triangles may indicate unusual connectivity patterns.

However, the raw triangle count is strongly affected by network size and density. Larger or denser networks naturally tend to contain more triangles. Therefore, it is often more informative to use a normalized measure such as the clustering coefficient or transitivity:

$$Transitivity = \frac{3 \times number\ of\ triangles}{number\ of\ connected\ triples}$$

Triangle counting shows how often relationships form tightly connected groups, while normalized measures make comparisons across networks more meaningful.

Key Concept #5. The fraction of closed triangles

The *fraction of closed triangles* measures how often an open three-vertex pattern is completed to form a triangle. It is also commonly called transitivity or the global clustering coefficient.

A connected triple consists of three vertices connected by at least two edges:
If there are two edges, (A, B), and (B, C), it becomes a closed triple when the third edge is present: (A, C).
The fraction is calculated as,

$$Fraction\ of\ closed\ triangles = \frac{3T}{\sum_{v \in V} \binom{d(v)}{2}}$$

Where

- T is the number of triangles
- d(v) is the degree of vertex v
- $\sum_{v \in V} \binom{d(v)}{2}$ is the number of connected triples

The factor of 3 appears because each triangle closes one triple centered at each of its three vertices.

For the DBLP network, the reported fraction is 0.1283, meaning that approximately 12.83% of connected triples are closed. In practical terms, when an author has collaborated with two other authors, those two coauthors have also collaborated with each other in about 12.83% of such configurations.

This value differs from the average clustering coefficient because transitivity combines all triples across the entire graph, giving greater weight to high-degree vertices. The average clustering coefficient first calculates a separate value for each vertex and then averages those values.

3. Dataset

The DBLP provides a comprehensive list of research papers in computer science. It started in 1993 at the University of Trier in Germany, and it is now operated by the Schloss Dagstuhl – Leibniz Center for Informatics. It indexes over 8 million publications from millions of authors, tracking major peer-reviewed journals and international conference proceedings.

This assignment includes the DBLP scientific collaboration network where nodes represent authors and edges connect authors that have co-authored a paper. Communities in a scientific domain correspond to people working in common areas and subareas of science.

We will use a curated version of the DBLP network available at the Stanford SNAP graph data dissemination site [2]. Note that the referenced page includes three files. The relevant file is available on the Canvas assignment page. Please use the data file provided there.

4. Execution environment and constraints

You will first need to install and configure a Spark cluster on the assigned Linux cluster in CSB. Your first two lab sessions will cover the installation process and the execution of a simple example. For this assignment, please use the same cluster configuration discussed during the lab sessions.

This programming assignment must be implemented exclusively using Apache Spark RDDs. Other APIs, such as DataFrames and Spark SQL, are not permitted. Third-party APIs and Spark libraries such as GraphX are also prohibited. If your submission does not meet these requirements, you will receive a score of zero.

There is no specific programming-language requirement for this assignment, except that R is not permitted. You may use PySpark, Java, or Scala. R is excluded because its limited support may create programming challenges later in the assignment.

This programming assignment emphasizes your proficiency in a cluster-computing environment. Therefore, your software must run correctly on your Apache Spark cluster. If it works correctly only on a single node, a significant number of points will be deducted. For the detailed grading rubric, please refer to Section 7.

5. Required tasks and deliverables

All listed tasks must satisfy the constrains specified in Section 4.

Task 1. Graph and community infrastructure

Implement a Spark application to,

a. Load the undirected edge list from the provided file into an RDD.

b. Identify and remove any self-loops.

c. Identify and remove any duplicate edges or vertices.

Task 2. Explore and characterize the graph

Implement Spark applications to calculate and report:

a. The number of unique nodes and edges.

b. The total number of nodes that belong to the single largest interconnected group (Weakly Connected Component) in this network graph

c. The total number of edges that belong to the single largest interconnected group (Weakly Connected Component) in this network graph

d. Average clustering coefficient

e. The count of triangles

f. Fraction of closed triangles

Your grading interview may include questions about:

1. Implementation details.
2. The concepts used to complete the tasks above and their implications for the DBLP co-authorship network.
3. A demonstration of your applications.
4. Challenges encountered and potential improvements to your implementation.

Bonus Task (1 percent of your course grade)
Calculate and report the diameter (longest shortest path) of this network

6. Minimum correctness check

Most of the correct answers are available in the web page of the provided dataset [2]. You can compare your answers with listed statistics for the graph.

7. AI usage guideline

You may use AI agents to assist with coding. We will provide guidelines for using them to generate better code while enhancing your learning.

8. Grading rubrics

	Total percentage towards the course grade	Requirements - Detailed itemized rubrics for partial scores will be posted in Canvas
Completeness of your code	1%	Your code must be executed correctly on a Apache Spark cluster in CSB with at least 2 Linux nodes. Your output should be correct.
Task 1	1%	In the grading interview, you should be able to answer all the questions correctly.
Task 2	3%	In the grading interview, you should be able to answer all the questions correctly.
Bonus task	1%	In the grading interview, you should be able to answer all the questions correctly.

The late submission policy is available in the course syllabus.

References

[1] Yang, Jaewon, and Jure Leskovec. "Defining and evaluating network communities based on ground-truth." In Proceedings of the ACM SIGKDD workshop on mining data semantics, pp. 1-8. 2012.

[2] Stanford Network Analysis Project. (n.d.). *DBLP collaboration network and ground-truth communities*. Stanford University. Retrieved August 31, 2026, from https://snap.stanford.edu/data/com-DBLP.html